

Transport Layer Theory Notes - Chapter 3

1. Transport Layer Overview

1.1 Introduction and Goals

The transport layer provides logical communication between application processes running on different hosts. Unlike the network layer which provides logical communication between hosts, the transport layer focuses on process-to-process communication.

Two Main Transport Protocols:

- **TCP (Transmission Control Protocol):** Connection-oriented, reliable transport
- **UDP (User Datagram Protocol):** Connectionless transport

1.2 Transport Layer Actions

Sender Side (Sending data):

- Gets messages from the application (like WhatsApp, browser, etc.).
- Decides what extra information (header) needs to be added, based on the transport protocol (TCP/UDP).
- Makes a “segment” by adding this header in front of the data.
- Sends the segment to the network layer so it can travel across the internet.

Receiver Side (Receiving data):

- Gets the segment from the network layer.
- Reads the header to check for errors and to follow protocol rules.
- Removes the header and takes out the original application message.
- Sends the message to the correct application (like the right chat window or video stream) using sockets.

2. Transport vs Network Layer Services

2.1 Key Distinctions

Network Layer:

- Logical communication between hosts
- Best-effort delivery service
- No guarantees about reliability, ordering, or flow control

Transport Layer:

- Logical communication between processes
- Relies on and enhances network layer services

- Can provide reliability, flow control, and congestion control
- Adds end-to-end functionality that network layer cannot provide

3. Internet Transport Protocols

3.1 TCP (Transmission Control Protocol)

Characteristics:

- **Connection needed** → Before sending data, sender and receiver first “shake hands” (setup a connection).
- **Reliable** → Makes sure all data reaches the receiver correctly.
- **In order** → Data arrives in the same sequence it was sent.
- **Congestion control** → Slows down sending speed if the network is busy.
- **Flow control** → Prevents sender from sending too much data at once (avoids receiver overload).
- **Setup delay** → Takes extra time (RTT) to start the connection.

Services provided:

- Guarantees data delivery using **ACKs (acknowledgments)** and **retransmissions**.
- Keeps data in the **right order**.
- Uses **flow control** so the receiver is never flooded.
- Uses **congestion control** so the network doesn't get overloaded.

Example: **Web browsing, emails, file downloads, video calls (when reliability is needed).**

3.2 UDP (User Datagram Protocol)

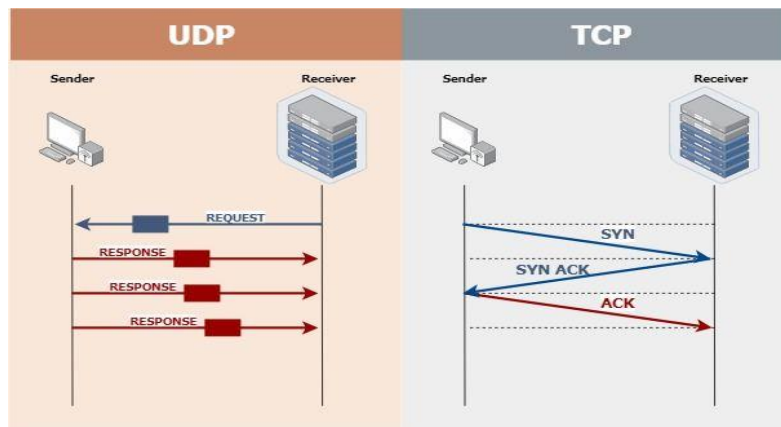
Characteristics:

- **No connection** → Just sends data, no handshake needed.
- **Unreliable** → No guarantee data will arrive, and order is not maintained.
- **Stateless** → Sender and receiver don't keep extra info about the connection.
- **Lightweight** → Small header size, less overhead.
- **No congestion control** → Can send data as fast as possible.

Advantages:

- **Fast** → No connection setup delay.
- **Simple** → Easy to implement, less complex than TCP.
- **Works even if network is unstable.**
- Good for apps where **some data loss is okay**.

Example: **Online gaming, live video streaming, voice calls, DNS lookups.**



3.3 Services Not Available

Neither TCP nor UDP provide:

- Delay guarantees for time-sensitive applications
- Bandwidth guarantees for consistent throughput requirements

4.1 Fundamental Concepts

Multiplexing (at Sender):

- Multiplexing means the transport layer takes data from many different applications (through their sockets) and prepares them to be sent over the network.
- It adds special information in the **transport header** (like port numbers) so that the receiver can later separate the data correctly.
- This allows many applications (like browser, WhatsApp, YouTube) to use the same network connection at the same time.

Demultiplexing (at Receiver):

- Demultiplexing means the transport layer at the receiver side checks the header information and sends each segment to the correct socket.
- This ensures that the right data goes to the right application (for example, your email data goes to Gmail, and your chat message goes to WhatsApp).
- It is very important because it keeps applications separate and avoids mixing up their data.

4.2 How Demultiplexing Works

Basic Mechanism:

- The receiver's host gets IP datagrams that contain transport-layer segments.
- Each datagram has the **source IP address** and **destination IP address**.

- Each transport segment inside it has the **source port number** and **destination port number**.
- The receiver uses these addresses and port numbers to figure out which application should get the data, and then delivers it to the correct socket.

Header Information Used:

- Source IP address
- Source port number
- Destination IP address
- Destination port number

Together, these four values are called a “4-tuple”. They uniquely identify each connection and help manage multiple active connections at the same time.

- **Multiplexing = combining data from many apps at the sender.**
- **Demultiplexing = separating data and delivering it to the correct app at the receiver.**

4.3 Connectionless Demultiplexing (UDP)

- **Creating a UDP socket:**
 - When you create a UDP socket, you must give it a **port number**.
 - Example: `DatagramSocket mySocket1 = new DatagramSocket(12534);` → This makes a socket that listens on port **12534**.
- **How data (datagrams) are handled:**
 - When a UDP packet (datagram) comes in, the computer looks at the **destination port number**.
 - It sends the datagram to the socket that matches that port.
 - If many datagrams have the **same destination port**, they all go to the **same socket**, even if they come from different sources.
- **Key idea:**
 - UDP only uses the **destination port number** to decide which socket to deliver data to.
 - It **doesn't care about the source IP or port**.
 - This makes it simple and fast, but there is **less control** compared to TCP.

4.4 Connection-Oriented Demultiplexing (TCP)

- **How a TCP socket is identified:**
 - A TCP socket is recognized by **4 values (4-tuple)**:
 1. Source IP

2. Source port
 3. Destination IP
 4. Destination port
- This means each client's connection is **unique**.
 - **How TCP demultiplexing works:**
 - The computer checks **all four values** in the incoming segment.
 - Even if many clients connect to the same server port (for example, port 80 for websites), each client gets a **separate socket**.
 - So their data never gets mixed up.
 - **Example:**
 - Three different clients connect to server **B** at port 80.
 - Each client has a different **source IP and source port**.
 - TCP creates **different sockets** for each client → the server can handle all of them at the same time.
 - **Key idea:**
 - TCP uses **both source and destination info** (IP + port) to separate connections.
 - This makes TCP powerful for handling **many users at once**.

5. UDP: User Datagram Protocol

5.1 UDP Characteristics

"No Frills" Protocol (UDP)

- **Basic idea:**
 - UDP is a very simple Internet transport protocol.
 - It gives only the **minimum service** – nothing extra.
- **Best effort service:**
 - UDP does **not guarantee delivery**.
 - A segment (packet) may get **lost**, or it may arrive **out of order**.
 - The application must handle such problems if needed.
- **Connectionless nature:**
 - No **handshake** before sending data (unlike TCP).
 - Each segment is treated **independently**.
 - No need to remember connection details between sender and receiver.

5.2 Why UDP Exists

Advantages of UDP

1. **Fast communication**
 - No handshaking or connection setup → less delay.
 - Suitable for real-time apps (like calls, video streaming, gaming).
2. **Low overhead**
 - UDP header is very small (only **8 bytes**) compared to TCP (20+ bytes).
 - Saves bandwidth.
3. **Supports broadcasting & multicasting**
 - Can send data to many receivers at once (useful in streaming, DNS, DHCP).
4. **Connectionless**
 - No need to maintain a connection state.
 - Works well for simple request-reply services (like DNS).
5. **Efficient for small data transfers**
 - Quick and simple for sending small amounts of data (like sensor updates, VoIP packets).

Trade-offs of UDP (or connectionless protocols)

- **No reliability guarantees:** UDP does not check if data reaches the receiver correctly. If reliability is needed, the application itself must handle errors or retransmissions.
- **No flow control:** UDP does not slow down the sender if the receiver is busy, so it can overwhelm the receiver with too much data at once.
- **No congestion control:** UDP does not monitor network traffic, so sending too fast can cause network congestion and packet loss.

5.3 UDP Applications

Common Uses:

- Streaming multimedia applications (loss tolerant, rate sensitive)
- DNS (Domain Name System) queries
- SNMP (Simple Network Management Protocol)
- HTTP/3 (latest HTTP version using QUIC over UDP)

Reliable Transfer over UDP (Application-Layer Enhancements)

- If an application needs **reliable data transfer** over UDP, it must **implement reliability itself**, because UDP does not provide it.
- The application can also add **congestion control** if it wants to avoid overloading the network.
- This approach allows applications to **customize reliability and control** mechanisms based on their specific needs, such as retransmitting lost packets or ensuring data arrives in order.

5.4 UDP Segment Structure

Header Fields (8 bytes total):

- Source port number (16 bits)
- Destination port number (16 bits)
- Length field indicating UDP segment size including header
- Checksum for error detection
- Application data (variable length payload)

5.5 UDP Checksum

Purpose:

- Detect errors (bit flips) in transmitted segments
- Provides basic data integrity checking

At the Sender (before sending):

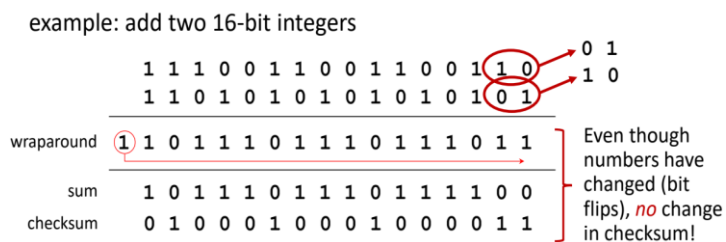
1. The sender takes the UDP segment (header + data).
2. It calculates a **checksum** → basically adding up all 16-bit words and taking the 1's complement.
3. The checksum value is placed in the UDP header.
4. The segment is then sent to the receiver.

At the Receiver (after receiving):

1. The receiver also adds up all 16-bit words of the segment **including the checksum**.
2. If the result is **all 1's (no error)** → the data is accepted.
3. If the result is **not all 1's (error detected)** → the segment is discarded.

Limitations:

- Internet checksum provides weak protection
- Some bit patterns can change without affecting checksum
- Additional error detection may be needed for critical applications



6. Principles of Reliable Data Transfer

6.1 Reliable Data Transfer Problem

Service Abstraction:

- Applications (upper layers) expect a reliable communication channel between the sending and receiving processes.
- The transport layer is responsible for providing this reliability, even though the underlying network layer may be unreliable.
- How complex this becomes depends on how unreliable the network is (e.g., frequent packet loss or errors).

Channel Problems:

1. **Packet loss:** Sometimes packets never reach the receiver. The transport layer must retransmit lost packets.
2. **Packet corruption:** Packets may get damaged during transmission, requiring error detection and possible correction.
3. **Packet reordering:** Packets can arrive out of order, so sequence numbers are needed to restore the correct order.
4. **Variable delays:** Network delays can vary, which makes it hard to decide when to retransmit a packet if an acknowledgment is not received.

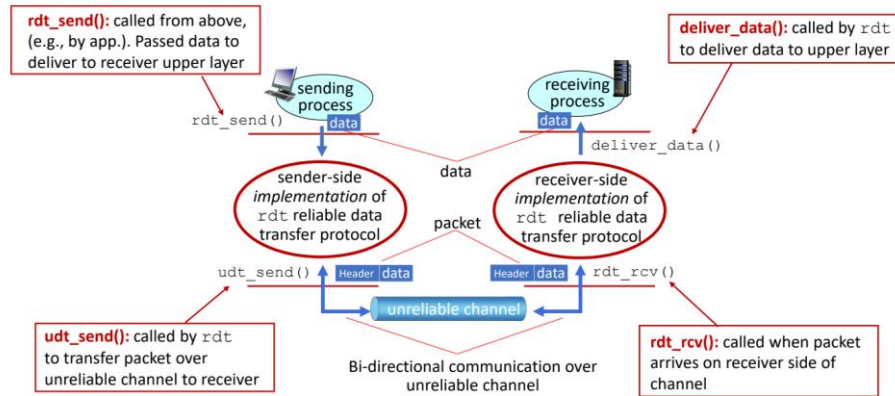
6.2 Reliable Data Transfer Interfaces

Sender Side:

- `rdt_send()`: called by application to send data
- `udt_send()`: called by RDT to transfer packet over unreliable channel

Receiver Side:

- `rdt_rcv()`: called when packet arrives from channel
- `deliver_data()`: called by RDT to deliver data to application



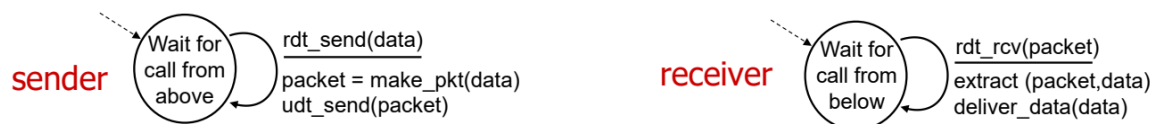
6.3 RDT 1.0: Reliable Channel

Assumptions:

- The communication channel is perfectly reliable.
- There are no bit errors in packets.
- There is no packet loss during transmission.

Implementation:

- The sender only needs to create a packet and send it through the channel.
- The receiver simply receives the packet, extracts the data, and delivers it to the application.
- Since the channel is perfect, no error detection or correction mechanisms are needed.
- This is the simplest case and is mainly used to demonstrate the basic structure of reliable data transfer (RDT).



6.4 RDT 2.0: Channel with Bit Errors

Problem:

- Underlying channel may flip bits in packets
- Need mechanisms to detect and recover from errors

Solutions:

- Checksum for error detection (e.g., Internet checksum)
- Acknowledgments (ACKs) for successful receipt confirmation
- Negative acknowledgments (NAKs) for error notification
- Retransmission on receipt of NAK

Stop-and-Wait Protocol:

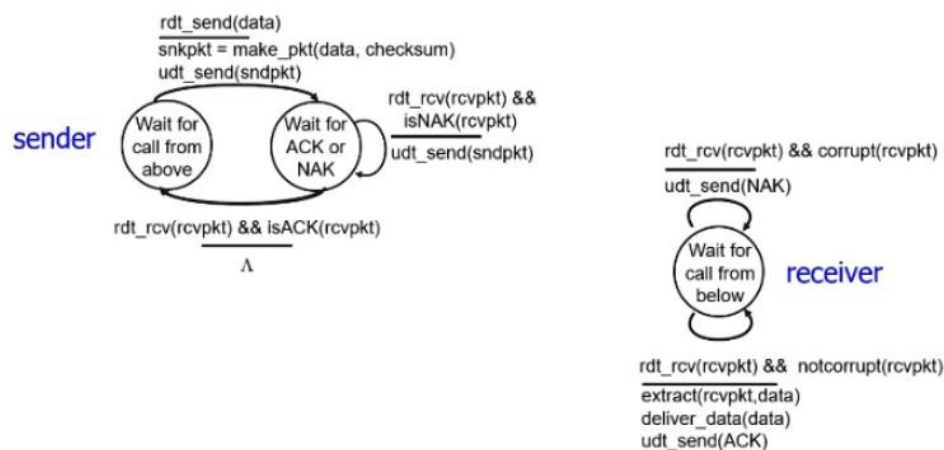
- Sender sends one packet then waits for receiver response
- Receiver sends ACK for correct packets, NAK for corrupted packets
- On receiving a NAK, the sender **retransmits the same packet**.

Fatal Flaw:

- ACK and NAK packets themselves can be **corrupted** during transmission.
- If this occurs, the sender cannot determine the receiver's actual state.
- A simple retransmission in such a situation can lead to **duplicate packets**, breaking reliability.

RDT 2.0 introduces error detection and recovery using **checksums, ACKs/NAKs, and retransmission**, but it suffers from the **fundamental flaw** that control messages (ACK/NAK) can themselves be corrupted.

rdt2.0: operation with no errors

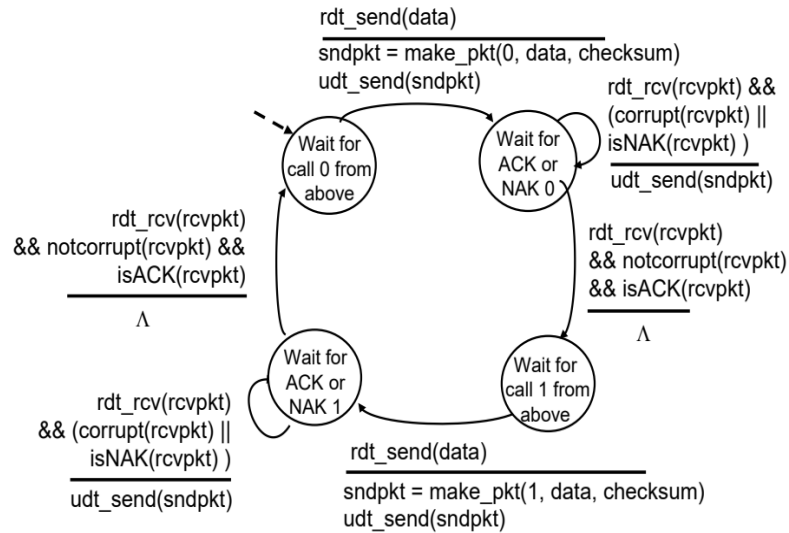


6.5 RDT 2.1: Handling Garbled ACK/NAKs

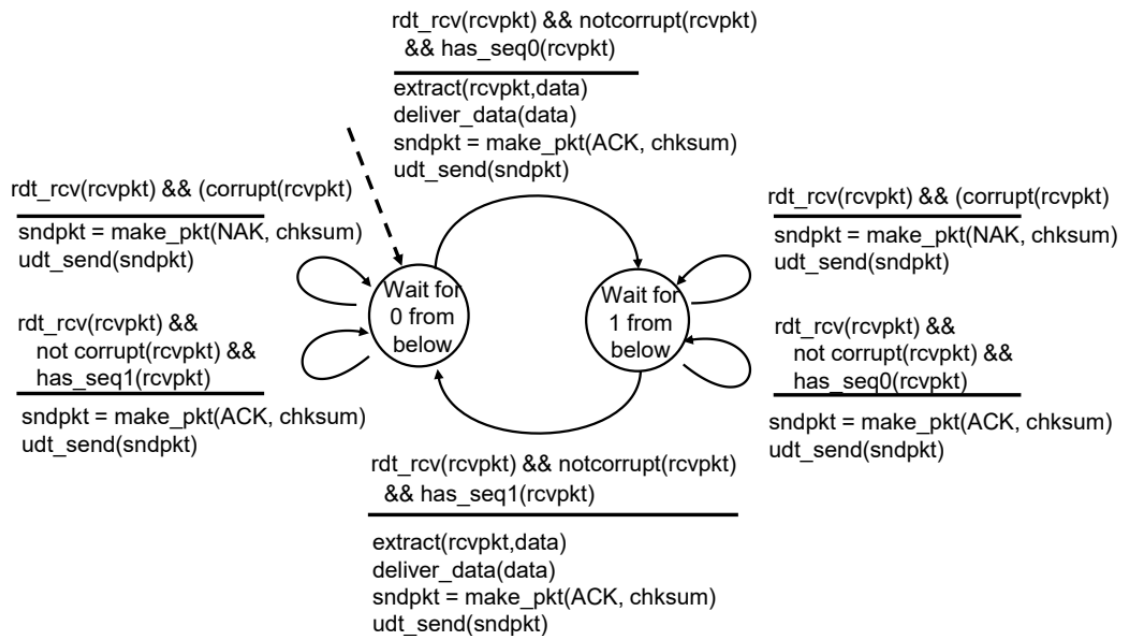
Solutions for Corrupted Control Messages:

- Sender adds sequence numbers to packets
- Two sequence numbers (0,1) sufficient for stop-and-wait
- Receiver discards duplicate packets
- Sender retransmits on corrupted ACK/NAK

Sender-----



Receiver-----



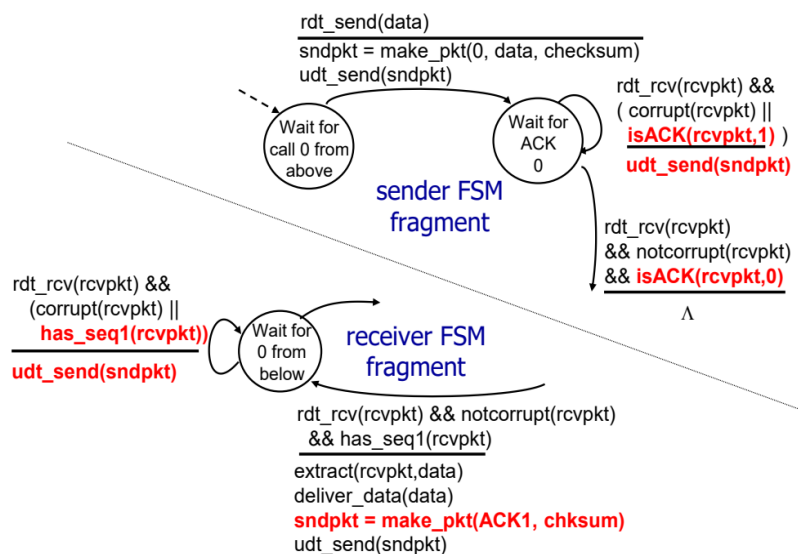
State Requirements:

- Sender must remember expected sequence number
- Receiver must track expected sequence number
- Both sides maintain state for duplicate detection

6.6 RDT 2.2: NAK-Free Protocol

Approach:

- Removes the need for NAK
- Use only ACKs, eliminate NAKs entirely
- Receiver sends ACK for last correctly received packet
- Duplicate ACK indicates error (equivalent to NAK)
- Sender retransmits on duplicate ACK receipt



Advantages:

- Simpler protocol with fewer message types
- TCP uses this NAK-free approach
- Reduces protocol complexity while maintaining functionality

6.7 RDT 3.0: Channels with Errors and Loss

New Problem:

- Channel can lose packets (data or ACKs) in addition to corrupting them
- Previous mechanisms insufficient for handling loss

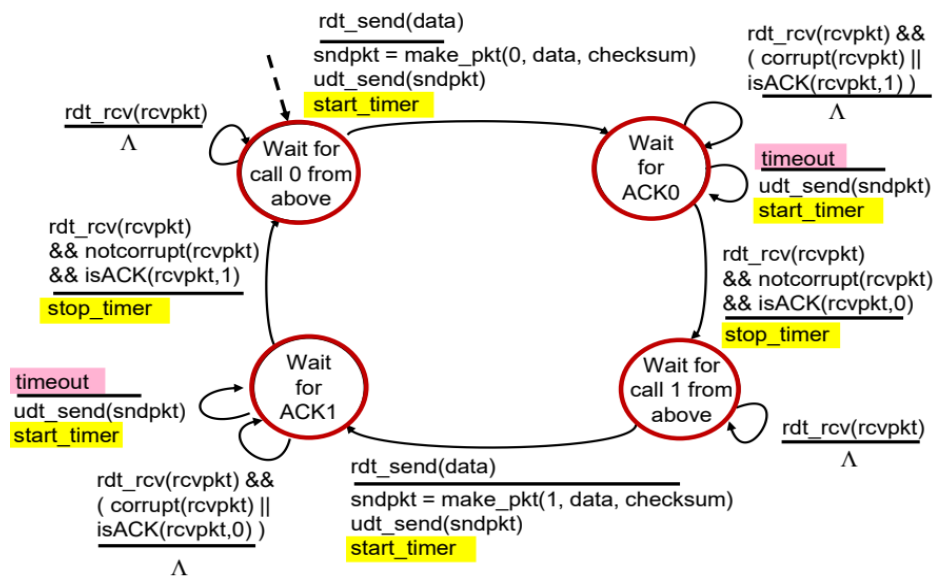
Solution Approach:

- Sender waits reasonable time for ACK (timeout mechanism)
- Retransmit if no ACK received within timeout period
- Handle delayed (not lost) packets and ACKs using sequence numbers

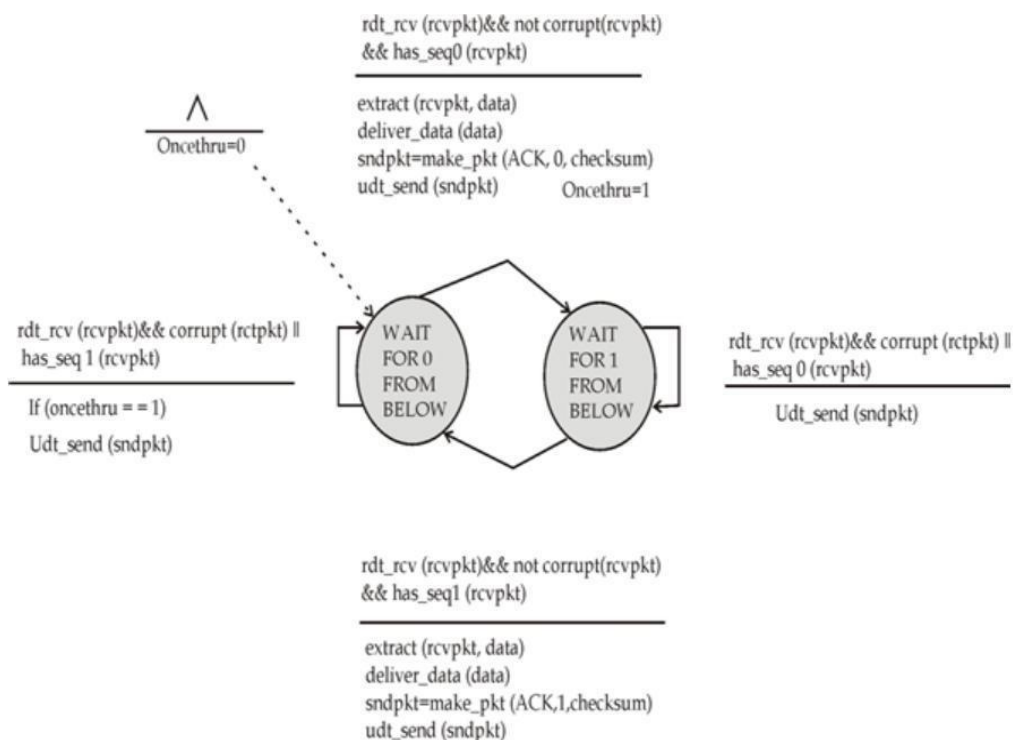
Timeout Considerations:

- The timeout should be set longer than the round-trip time (RTT), but not too long.
- If it is too short, the sender may think a packet is lost even when it isn't, leading to unnecessary retransmissions.
- If it is too long, the sender will take a lot of time to detect real packet loss, which slows down communication.

Sender-----

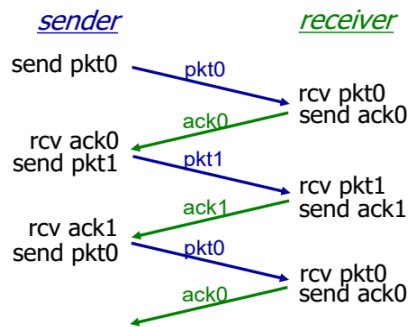


Receiver-----

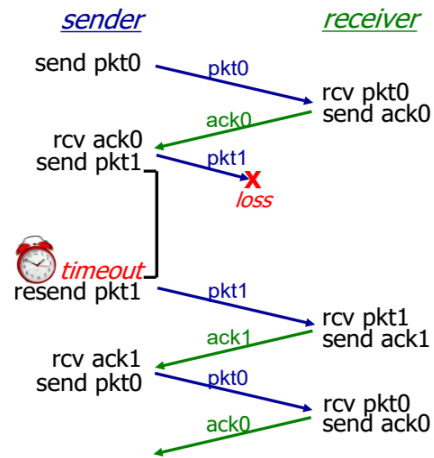


6.8 RDT 3.0 Performance Analysis

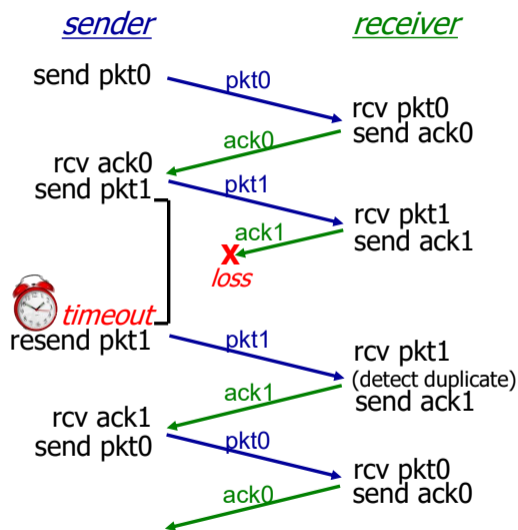
rdt3.0 in action



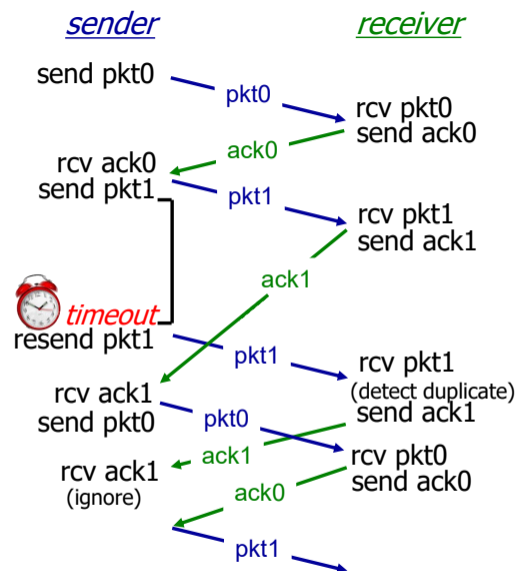
(a) no loss



(b) packet loss



(c) ACK loss



(d) premature timeout/ delayed ACK

RDT 3.0 & Pipelining

How it works

- Sender sends 1 packet and waits for an ACK before sending the next packet.
- Receiver sends an ACK when it correctly receives a packet.

Performance

- **Utilization** U_{sender} = fraction of time sender is busy transmitting:

$$U_{\text{sender}} = \frac{L/R}{RTT + L/R}$$

Where:

- L = packet size (bits)
- R = link speed (bits/sec)
- RTT = round-trip time

Example:

- L = 8,000 bits
- R = 1 Gbps
- RTT = 30 ms

$$D_{\text{trans}} = L/R = 8000/10^9 = 8\mu s$$

$$U_{\text{sender}} = \frac{0.000008}{0.03 + 0.000008} \approx 0.00027$$

Pipelined Protocols

- Send multiple packets without waiting for ACKs.
- Requires larger sequence numbers and buffering.
- Send N packets without waiting for ACKs.
- Utilization with window size N:

$$U_{\text{pipeline}} = \frac{N \cdot (L/R)}{RTT + (L/R)} \approx N \cdot U_{\text{SW}}$$

- Example: 3-packet pipeline $\rightarrow U_{\text{pipeline}} \approx 3 \times 0.00027 \approx 0.00081$
- Requires **larger sequence numbers** and **buffers**.

6.9 Pipelined Protocols

Concept

- In pipelining, the sender can send **more than one packet at a time** without waiting for each acknowledgment.
- This keeps multiple packets “in flight” (on the way) at once.
- It makes much better use of the network compared to **Stop-and-Wait**, where only one packet is sent at a time.

- To support this, we need a **larger sequence number range** and **temporary storage (buffering)** at sender/receiver.
- Error handling becomes **more complex**, because now multiple packets can be lost, corrupted, or delayed.

Benefits

- If 3 packets are sent in a pipeline, the channel is used **3 times more efficiently** than Stop-and-Wait.
- Makes better use of the available **network bandwidth**.

7. Pipelined Reliable Data Transfer Protocols

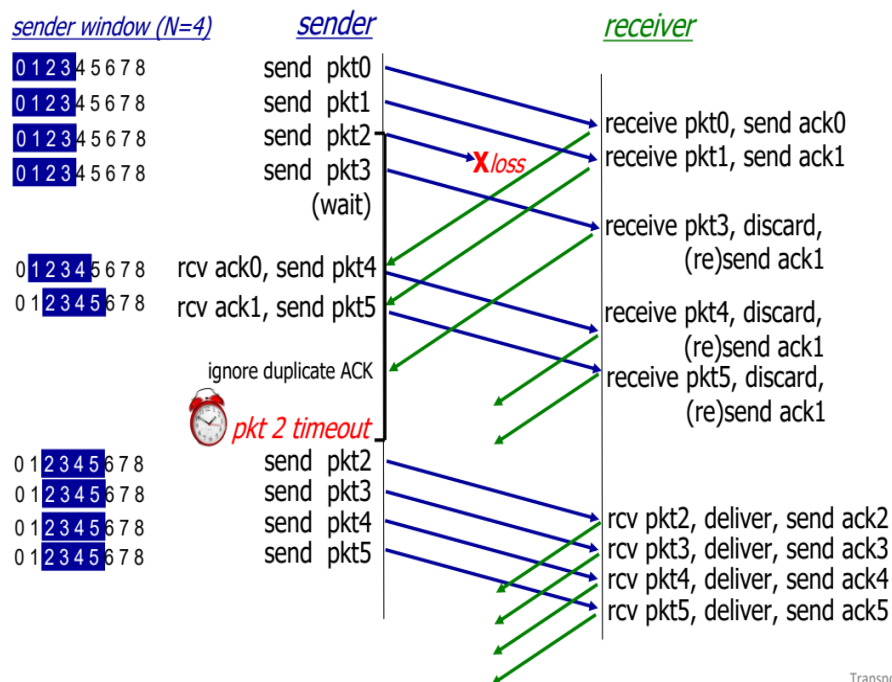
7.1 Go-Back-N (GBN)

Sender Characteristics

- The sender can send up to **N packets** at a time without waiting for ACKs.
- Each packet has a **k-bit sequence number** to identify it.
- The sender uses **cumulative ACKs**: when ACK(n) comes, it means all packets up to n are received.
- The sender keeps **one timer** for the oldest unacknowledged packet.

Window Management

- When the sender gets **ACK(n)**, the window moves forward and starts from n+1.
- If a **timeout** happens for packet n, the sender **re-sends packet n and all later packets** in the window.
- As ACKs arrive, the window keeps sliding forward.



Transport

Receiver Characteristics

- The receiver only sends **ACKs** (no negative ACKs).
- It always sends an ACK for the **last packet received in order**.
- It may send **duplicate ACKs** to help the sender.
- The receiver only needs to remember the **first packet number it is expecting (rcv_base)**.

Out-of-Order Handling

- If packets come **out of order**, the receiver can just **throw them away** (depending on design).
- It then re-sends an ACK for the **last in-order packet** it got.
- This makes the receiver simpler, but sometimes **wastes bandwidth** since out-of-order packets may need to be sent again.

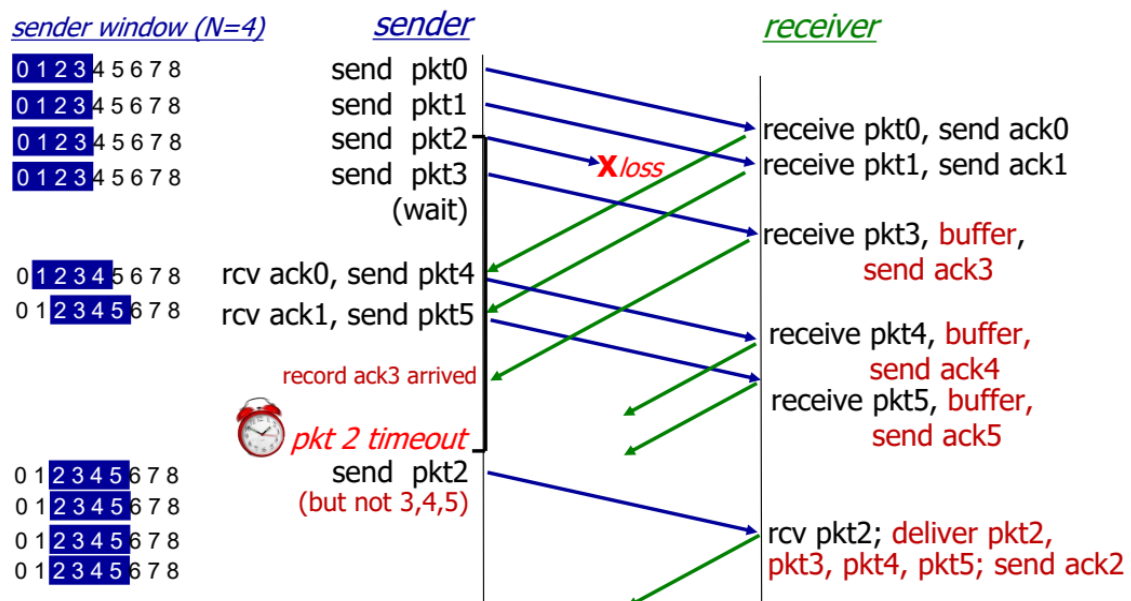
7.2 Selective Repeat (SR)

Sender Characteristics

- Sender gets **individual ACKs** for each packet.
- Keeps a **separate timer for every unacknowledged packet**.
- Has a **window of size N** → can send N packets without waiting.
- If a packet times out (ACK not received), **only that packet** is resent.

Receiver Characteristics

- Can **store (buffer)** packets that arrive out-of-order.
- Sends an **ACK for every correctly received packet**.
- Has a **receive window** → accepts only packets inside this range.



7.3 Selective Repeat Dilemma

- SR uses **sequence numbers** that **wrap around** after reaching the maximum.(like clock)
- **Problem:** If the **window size is too large**, the receiver can confuse an **old packet** with a **new one**.

Example:

- Sequence numbers = 0–7, window size = 5.
- Sender sends 0,1,2,3,4 → receiver delivers them.
- Receiver now expects 5,6,7,0,1.
- If old packet 0 arrives late → mistaken as new packet → confusion.

Solution:

- Keep **window size $\leq$ half of sequence number space**.
- Example: sequence numbers 0–7 → max window size = 4.

Concept

- **Reliable Data Transfer** ensures data is delivered correctly, in order, without loss or duplication.
- **Sequence numbers** are crucial to detect duplicates and maintain order.
- **Checksums** detect corruption.
- **Timers** handle packet loss.
- RDT evolution:
 - RDT 1.0 → Reliable channel.
 - RDT 2.0 → Adds ACK/NAK for corruption.
 - RDT 2.1 → Adds sequence numbers.
 - RDT 2.2 → ACK-only.
 - RDT 3.0 → Handles packet loss with timers.
- **Stop-and-Wait** is simple but inefficient.
- **Pipelining (GBN, SR)** improves efficiency.
- **TCP** in practice is based on RDT 3.0 + Pipelining (like SR/GBN) + congestion control.

8. TCP: Transmission Control Protocol

1. Introduction

TCP (Transmission Control Protocol) is a **connection-oriented** and **reliable transport layer protocol** in the Internet Protocol Suite. It works on top of IP and ensures accurate, ordered, and error-free delivery of data. TCP is widely used in **web browsing (HTTP/HTTPS)**, **email (SMTP, IMAP, POP3)**, **file transfer (FTP)**, and **remote login (SSH, Telnet)**.

2. Features of TCP

1. **Connection-Oriented Service** – Uses **three-way handshake** before data transfer.
2. **Reliable Data Transfer** – Error detection, acknowledgment, and retransmission.
3. **Full Duplex Communication** – Data can flow both ways at the same time.
4. **Byte Stream Transmission** – Treats data as a continuous stream of bytes, splits into segments.
5. **Flow Control** – Prevents sender from overwhelming receiver.
6. **Congestion Control** – Prevents network overload by adjusting sending rate.

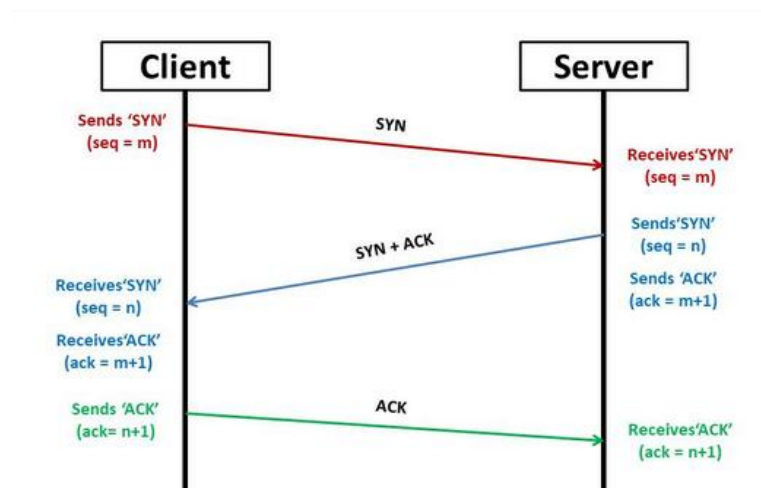
3. TCP Header Format (20–60 bytes)

- **Source Port (16 bits):** Sender's port.
- **Destination Port (16 bits):** Receiver's port.
- **Sequence Number (32 bits):** Position of first byte in the segment.
- **Acknowledgment Number (32 bits):** Next expected byte.
- **Header Length (4 bits):** TCP header size.
- **Flags (6 bits):** URG, ACK, PSH, RST, SYN, FIN.
- **Window Size (16 bits):** Flow control info.
- **Checksum (16 bits):** Error checking.
- **Urgent Pointer (16 bits):** Urgent data marker.
- **Options (variable):** MSS, timestamp, etc.

4. TCP Connection Establishment (Three-Way Handshake)

Three-Way Handshake: TCP uses a three-way handshake to make sure both client and server are ready, synchronized, and agree on sequence numbers, solving the problems that a simpler two-way handshake would have.

- Steps:
 - Client → Server: SYN** – Client requests a connection and sends its initial sequence number.
 - Server → Client: SYN-ACK** – Server acknowledges client's SYN and sends its own sequence number.
 - Client → Server: ACK** – Client acknowledges server's SYN. Now both sides know the connection is fully established.



5. TCP Connection Termination (Four Steps)

- Client sends **FIN**.
- Server replies with **ACK**.
- Server sends **FIN**.
- Client replies with **ACK** (connection closed).

6. Flow Control (Sliding Window Protocol)

- Ensures sender does not send more than receiver can handle.
- Receiver advertises a **window size (rwnd)** in ACK.
- Sender can send only up to rwnd bytes before waiting for acknowledgment.
- As ACKs arrive, window **slides forward**, allowing new data to be sent.

7. Congestion Control (AIMD Strategy)

Congestion control prevents **network overload**. TCP uses a **Congestion Window (cwnd)** that limits the amount of unacknowledged data in the network.

Phases:

1. **Slow Start** – cwnd starts with 1 MSS, grows exponentially until threshold (ssthresh).
2. **Congestion Avoidance (AIMD):**
 - **Additive Increase** – cwnd increases linearly (+1 MSS per RTT).
 - **Multiplicative Decrease** – On packet loss, cwnd is halved.
3. **Fast Retransmit & Fast Recovery** – Quickly retransmit lost packets and avoid restarting from slow start.

Balances throughput and congestion → stable network.

8.6 TCP Fast Retransmit

1. Meaning

Fast Retransmit is a **mechanism in TCP congestion control** used to quickly detect and retransmit lost packets **without waiting for a timeout**.

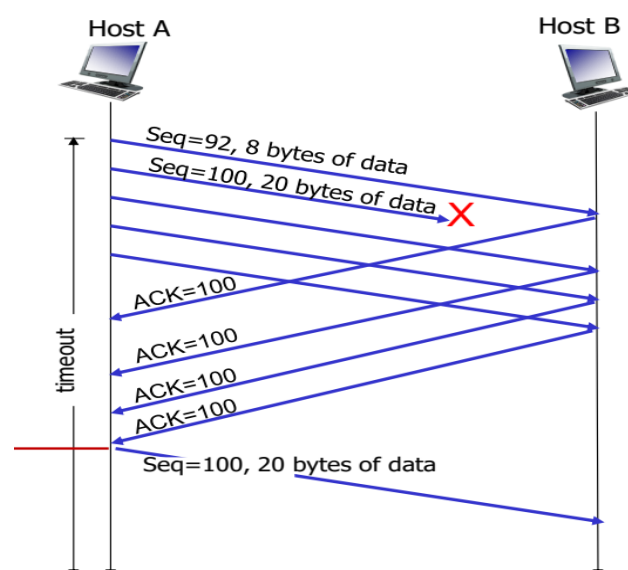
It improves performance by recovering from packet loss faster.

2. How it works

- Normally, TCP detects packet loss using a **timeout** (retransmission timer).
- But waiting for timeout increases delay.
- Instead, TCP uses **duplicate ACKs** to detect packet loss early.

Process:

1. If a segment is lost, receiver will keep acknowledging the last correctly received byte.
2. Sender receives **duplicate ACKs** (same ACK number multiple times).
3. After **3 duplicate ACKs**, sender assumes a packet is lost.
4. Sender immediately **retransmits the missing segment** → this is called **Fast Retransmit**.



8. Flow Control vs Congestion Control

Flow Control	Congestion Control
Protects receiver from being overwhelmed.	Protects network from being overloaded.
Uses sliding window with rwnd.	Uses congestion window (cwnd) and AIMD.
Based on receiver's capacity.	Based on network conditions (loss, delay).

9. TCP vs UDP

Feature	TCP	UDP
Connection	Connection-oriented	Connectionless
Reliability	Reliable (ACK, retransmit)	Unreliable
Ordering	Ordered delivery	No ordering
Speed	Slower (more overhead)	Faster (less overhead)
Use cases	Web, Email, FTP, SSH	DNS, VoIP, Games

13. Evolution of Transport Layer Functionality

13.1 Transport Protocol Evolution

Specific Challenges:

- Long, fat pipes: many in-flight packets, loss shuts down pipeline
- Wireless networks: loss from wireless issues treated as congestion
- Long-delay links: extremely long RTTs affect performance
- Data center networks: latency-sensitive applications
- Background traffic: low-priority flows need different handling

13.2 QUIC: Quick UDP Internet Connections

Architecture

- QUIC is an **application-layer protocol built on top of UDP**.
- It was **designed to make HTTP faster** (especially for web browsing, streaming, etc.).
- First developed by **Google** and used in their apps and servers.

Comparison with Traditional Stack

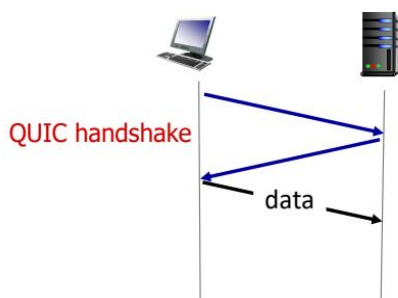
- **HTTP/2 over TCP:**
 - Uses TCP for reliability + TLS for security + HTTP for applications.
 - Each layer is separate.
- **HTTP/3 over QUIC:**
 - Combines everything into one protocol.
 - Reliability, security, and congestion control are integrated for better speed and performance.

Key Features

- **Multiplexing:** Can send many streams in a single connection.
- **Independent reliability & security:** Each stream is handled separately, so one problem doesn't stop others.
- **Common congestion control:** QUIC manages traffic for all streams together.
- **Built-in handshake:** Connection setup, encryption, and reliability are done together.

13.3 QUIC Connection Establishment

- **Traditional TCP + TLS:**
 - First: **TCP handshake** (to set reliability + congestion control).
 - Then: **TLS handshake** (to set authentication + encryption).
 - Needs **two handshakes**, so it's slower.
- **QUIC Handshake:**
 - Everything (reliability, congestion control, authentication, encryption) is done in a **single handshake**.
 - Needs only **1 round-trip time (RTT)**.
 - Much **faster to start communication** than TCP.



13.4 QUIC Streams and Head-of-Line Blocking

- **Problem in HTTP/2 over TCP:**
 - All streams are sent inside **one TCP connection**.
 - If one packet is lost → all streams stop until it's retransmitted.
 - This is called **head-of-line (HOL) blocking**.
- **QUIC's Solution:**
 - Each stream is **independent**.
 - If one stream has packet loss, only that stream is delayed.
 - Other streams **continue without waiting**.
 - Still, all streams share a **common congestion control** for fairness.